

PASTTEL: Parallel analysis framework for Termination and non-Termination of Lasso programs

Anissa Kheireddine^{1,2}[0000–0002–5958–4069], Souheib Baarir^{2,3}[0000–0001–8140–0273], and Hugo De Sa Pereira Pinto¹

¹ Dowers, Paris, France

`first.last@dowers.finance`

² Sorbonne University, LIP6, Paris, France

³ Sorbonne Paris North University, LIPN, France

`first.last@lipn.univ-paris13.fr`

Abstract. Proving termination or non-termination of *lasso programs* is a challenging problem in program verification. To unify state-of-the-art approaches under a common execution framework, we present PASTTEL, a modular and generic parallel portfolio framework for termination and non-termination analysis of lasso programs. PASTTEL is designed to: (1) facilitate the integration of new analysis algorithms into the portfolio, (2) execute registered strategies concurrently, and (3) act as a self-contained library component that can be seamlessly embedded into any external project requiring (non-)termination analysis. Initial experiments demonstrate that an instantiation of PASTTEL performs competitively against state-of-the-art tools.

1 Introduction

Proving the termination of a program is one of the most fundamental correctness properties in computer science, and is known to be undecidable in the general case. A practical way to approach this problem is to search for explicit termination or non-termination arguments. The existence of a certain termination argument — such as a linear ranking function — is decidable [5,7,26,30,35] and implies termination. However, if no linear ranking function can be found, we cannot conclude non-termination. Conversely, the existence of a non-termination argument, e.g., a linear recurrence set [24] or a geometric non-termination argument [31], is decidable and implies non-termination, but a failure to find one cannot conclude termination.

These analyses are commonly applied to *lasso programs*: execution traces consisting of a single while loop preceded by straight-line code, whose control flow graph takes the characteristic shape of a lasso.

In typical use cases, a lasso program is extracted from a control flow graph to represent a candidate infinite execution path, as encountered in CEGAR-based

termination synthesis [13,14,15,19].

We introduce PASTTeL⁴, a parallel analysis framework for the termination and non-termination of lasso programs. PASTTeL unifies state-of-the-art techniques within a concurrent portfolio that combines both termination and non-termination approaches. Our main contributions are as follows:

- We propose a modular and generic framework for implementing termination and non-termination strategies, designed to facilitate their integration into a unified portfolio.
- We implement several state-of-the-art constraint-based techniques, including ranking function synthesis [30] and geometric non-termination arguments [31].
- We evaluate an instance of our framework on the SV-COMP 2025⁵ benchmark suite, comparing its performance with the state-of-the-art ULTIMATE LASSORANKER⁶. The results show that the modularity and genericity of PASTTeL do not compromise performance, while its C++-based implementation yields a significant speedup in execution time.

Beyond its standalone use, PASTTeL is designed as a self-contained library that any external project can integrate to leverage (non-)termination analysis capabilities without knowledge of its internal design.

The remainder of this paper is organized as follows. Section 2 surveys the state-of-the-art termination and non-termination analysis tools and positions PASTTeL among them. Section 3 introduces the formal definition of lasso programs and describes the corresponding input format of PASTTeL. Section 4 describes the architecture of the framework and the concurrent orchestration of the analysis strategies. Section 5 showcases an instantiation of PASTTeL based on ULTIMATE LASSORANKER strategies. We conclude with an empirical evaluation of the tool in Section 6.

2 PASTTeL vs. the State-of-the-Art

To motivate the design of PASTTeL and highlight its originality, we first provide an overview of the state-of-the-art tools for (non-)termination synthesis.

Existing tools. Termination analysis is a very active field of research, and most tools operate at the whole-program level, iteratively extracting and analyzing individual lassos as part of their verification strategy. Among these, PROTON [32,33] integrates a fine-tuned LLM for ranking function synthesis, achieving a gold medal at SV-COMP2025⁵ in the Termination category. ULTIMATE

⁴ <https://github.com/Dowers/PaSTTeL/tree/artifact-vmcai27>

⁵ <https://sv-comp.sosy-lab.org/2025/>

⁶ https://www.ultimate-pa.org/lasso_ranker/

AUTOMIZER [19,25], the overall winner of SV-COMP 2025, implements a CEGAR loop combining ULTIMATE LASSORANKER with automata-theoretic techniques: termination arguments for individual lassos are generalized into Büchi automata representing all traces covered by the proof, which are then removed from the program via complementation and intersection. CPACHECKER [2] takes a different approach, extracting candidate lassos via predicate-based reachability analysis, but likewise delegates their (non-)termination analysis to ULTIMATE LASSORANKER. Beyond these, other whole-program tools such as APROVE [23,27], T2 [10], and VERYMAX [6,28] also address termination through their own dedicated strategies.

While the tools above operate at the program level, a smaller subset focuses specifically on single-lasso analysis, which is directly relevant to our work. ULTIMATE LASSORANKER [26,29,30] is a Java library integrated into ULTIMATE AUTOMIZER verification framework⁷. It extracts and analyzes linear lasso programs then provides termination proofs based on ranking function templates [29,30] (*affine*, *lexicographic*, *nested*, and *piecewise*) with linear inductive supporting invariants, and represents the state-of-the-art for single-lasso termination analysis over linear arithmetic. For non-termination, ULTIMATE LASSORANKER [31] synthesizes *Geometric Non-Termination Arguments* (GNTAs) for linear deterministic conjunctive lassos, applying *fixpoint* checking first and *GNTA* only if the former fails. Similarly, IRANKFINDER [5] synthesizes linear, lexicographic-linear, and multiphase-linear ranking functions for multi-path loops. On the non-termination side, LOAT [21,22] (Loop Acceleration Tool) proves non-termination via loop acceleration, computing closed-form summaries of loop iterations to detect reachable non-terminating behaviors. Finally, COMPACT [37,38] proves termination of linear loops through compositional abstraction using categorical deterministic affine transition systems.

Positioning PASTTEL among existing tools. The tools surveyed above share a common limitation: modularity is rarely a design goal, and adding a new termination or non-termination technique typically requires deep knowledge of the tool’s internals. Moreover, concurrent portfolio execution remains largely absent from existing tools. To our knowledge, APROVE is the only tool that comes close to this objective, orchestrating three backends concurrently: T2-based recurrent sets, LOAT-based acceleration, and direct SMT encoding. However, APROVE is not open-source, which limits both its reusability and extensibility.

While ULTIMATE LASSORANKER is open-source, its analyses are applied sequentially: on the termination side, ranking templates are tried one after another and on the non-termination side, *fixpoint* checking is attempted first, with *GNTA* executed only if the former fails. Furthermore, ULTIMATE LASSORANKER does not provide a modular interface for registering new strategies; its termination and non-termination analyses are tightly coupled to the internal architecture of the ULTIMATE framework, making standalone reuse non-trivial.

⁷ <https://www.ultimate-pa.org/automizer/>

PASTTEL is designed to address these limitations. It is an open-source, generic, and modular C++ framework that also exposes a library interface, enabling any external project to embed and leverage (non-)termination analysis capabilities as a self-contained component. New strategies can be integrated by implementing a single interface, making it straightforward to enrich the portfolio. Unlike APROVE, the portfolio is fully extensible. Unlike LASSORANKER, all registered strategies are executed concurrently, so that the first conclusive result — whether a termination or non-termination proof — is immediately returned without waiting for the remaining strategies to complete. Beyond the approaches currently integrated, the modular design of PASTTEL makes it straightforward to incorporate any existing technique. As a proof of concept, Section 5 presents an instantiation of PASTTEL based on the techniques underlying ULTIMATE LASSORANKER.

3 Lasso Program

A variety of techniques have been proposed for reasoning about lasso programs, including constraint-based methods [1,5,7,8,12,26,31,35], abstract interpretation [16,36], and algebraic approaches based on eigenvalue analysis [9]. As this work focuses primarily on constraint-based methods, we now introduce the formal definitions of (*linear*) lasso programs and the notion of termination considered throughout the paper.

We denote by $\mathbf{x} = (x_1, \dots, x_n)^\top \in \mathbb{R}^n$ the vector of n program variables, and by $\mathbf{x}' = (x'_1, \dots, x'_n)^\top \in \mathbb{R}^n$ the variables of the next state.

To abstract away from program syntax, we define a lasso program directly by the two relations that generate its execution sequences.

Definition 1 (Lasso Program [26,31]). *Given a set of states Σ , a lasso program $P = (\tau_{stem}, \tau_{loop})$ consists of two relations $\tau_{stem} \subseteq \Sigma \times \Sigma$ and $\tau_{loop} \subseteq \Sigma \times \Sigma$, called the stem and the loop of P , respectively.*

An execution of P is a possibly infinite sequence of states $\sigma_0, \sigma_1, \dots$ such that $(\sigma_0, \sigma_1) \in \tau_{stem}$ and $(\sigma_i, \sigma_{i+1}) \in \tau_{loop}$ for all $i \geq 1$. The program P is terminating if it admits no infinite execution.

A loop program is the special case where $\tau_{stem} \equiv true$.

Linear Arithmetic. The expression $A \cdot \mathbf{x} \leq \mathbf{b}$ denotes the conjunction of linear constraints with $(n, m) \in \mathbb{N}^2$, $A \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$,

$$\bigwedge_{j=0}^m \left(\sum_{i=0}^n a_{ij} \cdot x_i \right) \leq b_j.$$

Linear relation. A linear relation τ , is a relation defined by a conjunction of linear inequalities over $\mathbf{x}$ and $\mathbf{x}'$ with $(n, m) \in \mathbb{N}^2$, $A \in \mathbb{R}^{m \times 2n}$ and $\mathbf{b} \in \mathbb{R}^m$:

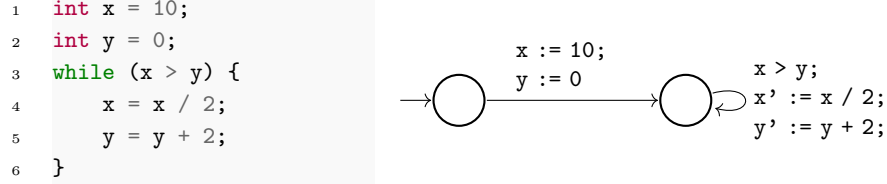


Fig. 1: The control flow graph of the left-side lasso program is defined by $\tau_{\text{stem}} = x := 10; y := 0$ and $\tau_{\text{loop}} = x > y; x' := x / 2; y' := y + 2;$

$$\tau = \{(\mathbf{x}, \mathbf{x}') \in \mathbb{R}^n \times \mathbb{R}^n \mid A \begin{pmatrix} \mathbf{x} \\ \mathbf{x}' \end{pmatrix} \leq \mathbf{b}\}$$

Definition 2 (Linear Lasso Program [26,31]). A linear lasso program is a pair $P = (\tau_{\text{stem}}, \tau_{\text{loop}})$ where $\tau_{\text{stem}}, \tau_{\text{loop}} \subseteq \mathbb{R}^n \times \mathbb{R}^n$ are linear relations over the current-state variables $\mathbf{x}$ and next-state variables $\mathbf{x}'$.

Figure 1 illustrates these definitions on a concrete example. The program in left-side of Figure 1 consists of a straight-line initialization followed by a single while loop. Its formal representation in right-side of Figure 1 exhibits the characteristic lasso shape: a stem encoding the initial assignments $x := 10; y := 0$, and a loop body encoding the transition $x' := x / 2; y' := y + 2;$ guarded by $x > y$.

Linearization of Lasso Programs

Some (non-)termination analysis strategies operate exclusively on linear lasso programs [26,29,30,31]. When the input lasso contains non-linear constructs (Boolean variables, division, modulo, ...), a pre-processing linearization phase is initiated⁶. For instance, Boolean variables are replaced by integer comparisons over a fresh auxiliary variable v , where *true* is encoded as $v \geq 1$ and *false* as $v \leq 0$.

When full linearization is not achievable — for instance, when an instruction translating a product of two free variables $x \times y$ appears — three strategies may be adopted. First, *over-approximation*: the non-linear constraint is replaced by $\top$, making the constraint trivially satisfied; termination analyses may then become too permissive and fail to produce a witness. Second, *under-approximation*: the constraint is replaced by $\perp$, making it unsatisfiable; non-termination analyses may then fail to retrieve a valid non-terminating execution. Third, an *exception* is raised to signal that linearization could not be completed. All three strategies are implemented in PASTTEL, and the desired behavior can be specified via a configuration parameter.

4 Architecture of the Framework

The performance of existing termination and non-termination analysis strategies heavily depends on implementation choices such as the programming language or the libraries used (*e.g.*, C++ vs. JAVA). This makes it difficult to

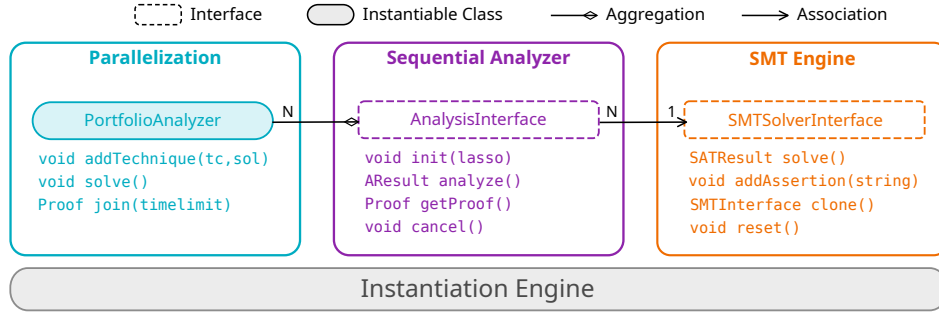


Fig. 2: The architecture of PASTTEL

compare strategies fairly without introducing a technological bias, or at best, fully reimplementing them to eliminate it. Moreover, integrating new strategies on top of existing ones remains a non-trivial task.

PASTTEL is designed to address these issues. The architecture of PASTTEL is structured around three core components: a sequential analyzer (**AnalysisInterface**); an SMT engine (**SMTSolverInterface**); and a parallelization layer (**PortfolioAnalyzer**), as illustrated in Figure 2). Any instantiation of the framework must provide concrete implementations of these three components.

SMT Engine: SMTSolverInterface. Constraint-based strategies reduce the synthesis of a (non-)termination argument to a satisfiability problem. This interface provides them with an SMT solver and allows different solvers to be plugged in interchangeably. PASTTEL currently supports Z3 [18] and CVC5 [3]. Each sequential analyzer holds its own solver instance, which it passes to its constraint-based strategies. The main methods are:

- **void addAssertion(string assertion):** adds a constraint to the solver.
- **SATResult solve():** checks the satisfiability of the current constraints.

Sequential Analyzer: AnalysisInterface. This adapter defines the basic functions expected from a sequential (non-)termination analyzer. The **LassoProgram** data structure used as input represents the lasso in linear format, with the stem and loop transitions, the list of program variables, and other information required for the analysis. The main methods of this interface are:

- **void init(LassoProgram* lasso):** initializes the analyzer with the input lasso program and apply (if needed) preprocessing step as linearization.
- **AnalysisResult analyze():** launches the analysis using the analyzer’s own SMT solver instance. Returns **TERMINATING**, **NON_TERMINATING**, or **UNKNOWN**.
- **ProofCertificate getProof():** returns a proof certificate for the (non-)termination result in plain text or JSON format.
- **void cancel():** interrupts the current search initiated by the caller.

Parallelization: PortfolioAnalyzer. This component manages the threads deployed for the concurrent execution of termination and non-termination strategies on the same input lasso program. Its main methods are:

- `void addTechnique(AnalysisInterface* technique, SMTSolverInterface solver)`: registers an analysis technique into the portfolio with its respective `solver` instance.
- `void solve()`: launches the concurrent search across all registered strategies.
- `ProofCertificate join(int timelimit)`: blocks until one technique returns a conclusive result or the time limit is reached. If the result differs from UNKNOWN, the corresponding proof certificate is returned.

Algorithm 1: init-PaSTTeL function

Input: *lasso*: linear lasso program, *n*: number of threads, *solverType*: {Z3, CVC5}

```

1 solvers ← Create n of solverType from SMTSolverInterface;
2 portfolio ← Create PortfolioAnalyzer(lasso, n);
3 portfolio.addTechnique(new TerminationSA(...), solvers[0]);
4 portfolio.addTechnique(new NonTerminationSA(...), solvers[1]);
5 portfolio.addTechnique(...) ...;
6 portfolio.solve();
7 proof ← portfolio.join(timelimit);
8 print(proof);
```

Engine Instantiation: init-PaSTTeL. To create a particular instance of PaSTTeL, the user has to adapt the `init-PaSTTeL` function (Algorithm 1), which serves as the entry point for instantiating and binding all components of the framework. Its parameters expose the key configuration points, allowing any external tool to embed PaSTTeL as a self-contained library with the desired configuration.

First, the *n* concrete solver instances of type *solverType* (inheriting from `SMTSolverInterface`) are instantiated (line 1). The portfolio analyzer is then instantiated with a reference to the *lasso* structure (line 2). Analysis strategies (inherited from `AnalysisInterface`) are registered into the portfolio (lines 3–5), each linked to a dedicated solver instance. The concurrent search is initiated via `solve` (line 6), and the main thread blocks until a conclusive result is found or the time limit is reached (line 7). The final result is retrieved, along with the proof certificate if one was produced (line 8).

5 PaSTTeL "à la ULTIMATE LASSORANKER"

As a proof of concept to validate the generic aspect of our framework, we selected the techniques underlying ULTIMATE LASSORANKER (ULR), the lasso-level analysis library of ULTIMATE AUTOMIZER, overall winner of SV-COMP 2025. We implemented P-ULR, an instantiation of PaSTTeL that replicates the core (non-)termination strategies of ULR. Each technique is realized as an independent analyzer registered into the portfolio and executed concurrently with the others. The resulting instantiation is illustrated in Figure 3.

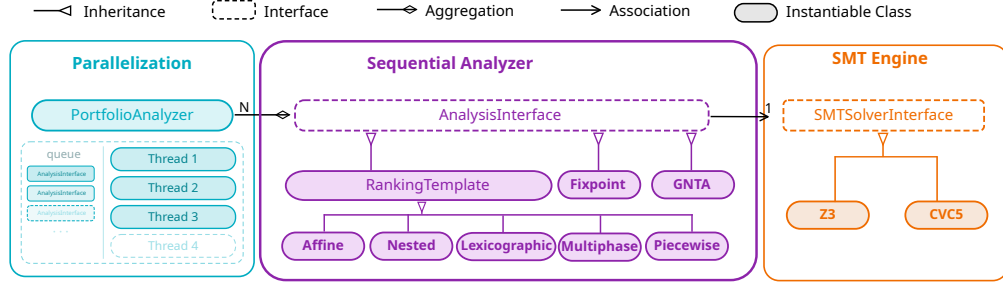


Fig. 3: P-ULR instantiation architecture

```

1 {
2   "program_vars": [
3     {
4       "x": "Int", "y": "Int"
5     },
6   ],
7   "stem": [
8     {
9       "formula": "(and (= x_0 10) (= y_0 0))",
10      "in_vars": {},
11      "out_vars": { "x": "x_0", "y": "y_0" }
12    },
13  ],
14  "loop": [
15    {
16      "formula": "(and (> x_1 y_1) (= x_2 (div x_1 2)) (= y_2 (+ y_1 2)))",
17      "in_vars": { "x": "x_1", "y": "y_1" },
18      "out_vars": { "x": "x_2", "y": "y_2" }
19    }
20  ]
21 }

```

Fig. 4: Input format of lasso program in Figure 1

Input Format & Linearization. While PASTTEL operates directly on linear lasso program structures (see Section 4), P-ULR provides a JSON-based input format (see Figure 4) as a convenience layer, designed to be generic and tool-agnostic. Formulas are encoded following the SMT-LIB standard [4], with support for Single Static Assignment (SSA) form [17]. Each input file specifies the program variables and their types, together with the input and output variables of both τ_{stem} and τ_{loop} ⁸.

P-ULR consists of approximately 10,000 lines of C++ code, built on top of a core framework of about 3,000 lines. Given a lasso program in the JSON-based SMT-LIB format, P-ULR first linearizes it when necessary, using a preprocessing phase that closely follows that of ULR, and then dispatches the analysis to the registered strategies. Since all implemented components inherit from the framework’s abstract interfaces, they are fully reusable and can be extended or complemented with additional techniques, such as new ranking function templates, without modifying the existing codebase.

Termination. P-ULR provides a constraint-based synthesis of termination arguments for linear lasso programs using linear ranking templates [29,30]. This

⁸ Support for additional fields, including axioms and uninterpreted functions, is currently under development.

approach generalizes constraint-based termination analysis by providing a unified framework for synthesizing ranking functions with affine-linear components, encompassing *nested*, *piecewise*, *parallel*, and *lexicographic* ranking functions, which can further be combined to form a more powerful composite templates. Rather than using Farkas’ lemma [20], the approach relies on Motzkin’s transposition theorem [34] for two reasons: (1) to transform the generated $\exists\forall$ -constraint into an $\exists$ -constraint directly solvable by an SMT solver, and (2) to handle ranking templates that require both strict and non-strict inequalities. The method is extended to the general setting of linear lasso programs by using affine-linear inductive invariants to extract from the stem the information that is crucial for the termination proof [7,11,26]. For instance, on the example of Figure 1, the synthesized ranking function is $f(x, y) = 2x - 2y - 1$, with supporting invariant $y + 2 \geq 0$.

Currently, P-ULR implements *affine*, *nested*, *lexicographic*, *multiphase*, and *piecewise* ranking templates, which can be run concurrently, whereas ULR tries them sequentially in a fixed order.

Non-Termination. P-ULR implements a constraint-based synthesis of non-termination arguments for linear lasso programs, integrating two approaches: *fixpoint* checking and *Geometric Non-Termination Arguments (GNTA)* [31]. A *GNTA* represents an infinite execution as a pointwise sum of geometric series: for $k \geq 2$, the k -th state is $\mathbf{x}_1 + \sum_{i=0}^{k-2} \lambda^i \cdot \mathbf{y}$, where $\mathbf{x}_1$ is the program state before the first loop iteration, $\mathbf{y}$ is the initial direction of the execution, and λ is the rate at which the execution keeps moving in that direction. This approach is complete for deterministic lassos with non-negative eigenvalues and reduces the search to an existential constraint in non-linear integer or real arithmetic (depending on the variable types), which can be discharged by an SMT solver. *Fixpoint* checking is the special case of *GNTA* where $\mathbf{y} = \mathbf{0}$: it searches for a state $\mathbf{x}$ such that $(\mathbf{x}, \mathbf{x}) \in \tau_{\text{loop}}$, i.e., a state from which the loop can iterate indefinitely without changing. As described in Section 2, ULR applies these two methods sequentially; P-ULR instead runs them concurrently, so that the *GNTA* analysis does not have to wait for the outcome of *fixpoint* checking.

Limitations. The current version of P-ULR has several limitations. First, the linearization step does not yet cover all non-linear cases: in particular, array operations such as constant arrays, array-valued `ite`, and array disequalities, as well as uninterpreted functions with quantified axioms, are not completely supported. Second, at the framework level, the `LassoProgram` structure currently provides better support for linear formulas; non-linear formulas are passed as-is to the registered strategies, which is valid for strategies that do not require linearization such as *fixpoint*. Third, at the strategy level, P-ULR currently supports linear rational arithmetic (QF_LRA) for termination synthesis and linear integer arithmetic (QF_LIA) for *GNTA* synthesis. Finally, simplification of synthesized ranking function coefficients and formula optimization have not yet been implemented.

Tool	#Terminating	Time (s)	#Non-Terminating	Time (s)
ULR-BASELINE		1173		20
P-ULR-SEQ	4141	102	1586	3
P-ULR-PAR7		47		3

Table 1: Comparison of P-ULR with ULR on **5727** lasso programs.

6 Numerical comparison

Experimental Setup. The experiments were conducted on a machine with 16 vCPUs of an Intel Core i5-14400T processor and 30 GiB of memory. We used 2856 programs: 1557 C programs drawn from the SV-COMP 2025⁵ and TERM-COMP 2025⁹ competitions, and 1299 Boogie programs from the ULTIMATE⁶ repository.

We use two time-outs: 3000 s per program for whole-program verification runs, and 600 s per lasso for lasso-level analyses. To build the benchmark, we ran ULTIMATE BÜCHI AUTOMIZER, which relies on ULR for lasso-level analysis, on each program and serialized to JSON every lasso submitted to ULR, yielding 5727 lassos. The time spent by ULR on each strategy was logged in the corresponding lasso file, and its total time on a lasso never exceeded 600 s. P-ULR was then evaluated on the same lassos under the per-lasso time-out.

Sequential Comparison. To ensure a fair comparison between P-ULR and ULR, we disabled the parallel execution of P-ULR and ran it in sequential mode, denoted P-ULR-SEQ. On the other hand ULR, renamed as ULR-BASELINE, executes its strategies in a specific static order: *fixpoint* checking then *GNTA* for non-terminating analysis, and *affine*, *nested*, *multiphase* and *lexicographic* for terminating analysis. P-ULR-SEQ follows that same sequential order in addition to *piecewise* as the last ranking template. Both tools were evaluated using Z3 (v 4.16.0) as the underlying SMT solver.

Table 1 reports the results of analyzing 5727 lasso programs, shown in the first two rows of the table. For each tool, we report the number of instances proved terminating and non-terminating, along with the accumulated solving time over the commonly solved instances. We discarded lassos that P-ULR-SEQ fails to prove due to linearization limitation. Overall, P-ULR-SEQ achieves a lower solving time than ULR-BASELINE, with a total gain of 1088 s (~18 minutes). Notably, the accumulated time for termination synthesis is reduced by a factor of 11, from 1173 s down to 102 s.

The scatter plots in Figures 5 illustrate the same results on a logarithmic time scale.

– On terminating instances, P-ULR-SEQ is faster than ULR-BASELINE on most lassos. This gain should be put into perspective, as most of the benchmark lassos admit an *affine* ranking function, so that the more expressive templates are rarely needed. The synthesized ranking functions may also differ, since ULR-BASELINE relies on optimizations not yet integrated in P-ULR; in particular, without the

⁹ <https://termination-portal.org/wiki/TPDB>

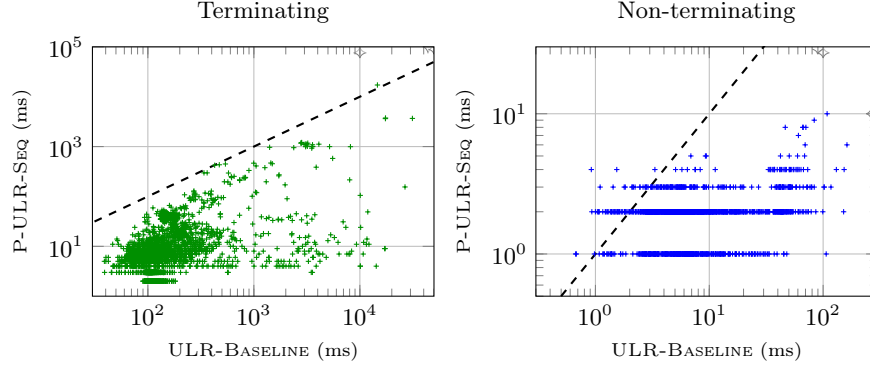


Fig. 5: Scatter plots of P-ULR-SEQ against ULR-BASELINE

partitioning step, P-ULR produces ranking functions with larger coefficients (Section 5).

– On non-terminating instances, two clusters appear: instances solved by *fixpoint* checking (bottom left), where both tools perform similarly, and instances requiring a *GNTA* (upper right), where P-ULR-SEQ is clearly faster.

Parallel Comparison. We now compare both tools in their full execution mode. P-ULR is run with 7 concurrent threads, referred to as P-ULR-PAR7, each running an independent `SequentialAnalyzer` instantiated with one of the following strategies: *fixpoint*, *GNTA*, and the *affine*, *nested*, *lexicographic*, *multiphase*, and *piecewise* ranking templates. The last four are parameterized by the number n of affine-linear functions (phases, components, or pieces) that jointly form the termination argument; since they reduce to the *affine* template for $n = 1$, their threads iterate over $n = 2, 3, 4$.

The results are reported in the third row of Table 1. P-ULR-PAR7 reduces the total execution time by a factor of 23, achieving a gain of 1143 seconds (~ 19 min) over ULR-BASELINE. However, compared to P-ULR-SEQ, the parallel execution yields little additional speedup. This is explained by the composition of the benchmark: 80% of lasso programs admit an *affine* ranking function, which P-ULR-SEQ already resolves first in its sequential order, leaving limited opportunity for parallelism to bring further improvement on this benchmark.

Integrate P-ULR inside Ultimate.

To assess P-ULR within a complete verification pipeline, we added to ULTIMATE BÜCHI AUTOMIZER a plugin of 520 lines of code that invokes P-ULR as an external (non)-termination analyzer. P-ULR acts as a first-line analyzer: it runs its techniques in parallel on each lasso to be proven (non)-terminating, and the analysis falls back to LASSORANKER whenever it fails, so that no capability of the original toolchain is lost.

We compared this configuration, ULTIMATE-PL, with the default one, ULTIMATE-LR, on 1087 C and Boogie programs from our benchmark (Figure 6), with a time-out of 1000 s per program, close to the 15-minute limit used in SV-COMP.

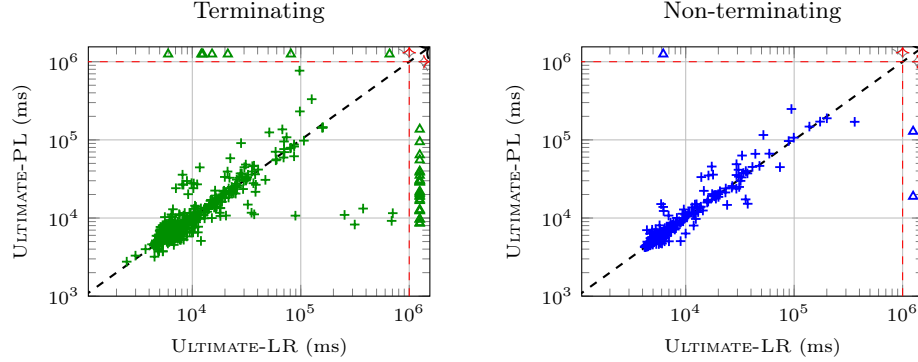


Fig. 6: Comparison of ULTIMATE-PL with ULTIMATE-LR on terminating (left) and non-terminating (right) programs. Programs solved by both tools are marked with +, those solved by only one tool with $\triangle$, the failing tool being placed beyond the timeout lines (red).

On the 883 programs solved by both, ULTIMATE-PL reduces the cumulative verification time from 13,491 s to 12,763 s (-5.4%), thanks to parallelism and the C++ implementation. This end-to-end gain is more modest than in isolation, since ranking function synthesis accounts for only part of the verification time. ULTIMATE-PL also solves 28 programs on which ULTIMATE-LR fails, while failing on 8 programs solved by ULTIMATE-LR, due to timeouts or unsupported operations of the ULTIMATE pipeline.

7 Conclusion

We presented PASTTEL, a modular and tool-agnostic framework that unifies state-of-the-art termination and non-termination analyses for linear lasso programs into a concurrent portfolio. As a proof of concept, we instantiated the framework with the techniques underlying ULTIMATE LASSORANKER, yielding P-ULR, which reduces the analysis time of lasso programs. We also plugged P-ULR into ULTIMATE as a first-line termination analyzer, with LASSORANKER as a fallback. This lightweight integration speeds up end-to-end verification, allows more programs to be solved, and shows that PASTTEL instantiations can be readily embedded into existing verifiers.

Several directions are planned as future work. First, we intend to extend the linearization pre-processing step to a broader class of programs, starting with the full set of lassos produced by ULTIMATE BÜCHI AUTOMIZER and progressively supporting additional constructs such as bit-vector arithmetic. Second, we plan to integrate recurrence set synthesis as an additional non-termination strategy. Finally, we aim to address the limitations identified in our evaluation, notably by integrating the partitioning step of ULR and by evaluating PASTTEL on benchmarks requiring more diverse termination arguments.

References

1. Roberto Bagnara, Fred Mesnard, Andrea Pescetti, and Enea Zaffanella. A new look at the automatic synthesis of linear ranking functions. *Information and Computation*, 215:47–67, 06 2012. doi:10.1016/j.ic.2012.03.003.
2. Daniel Baier, Dirk Beyer, Po-Chun Chien, Marek Jankola, Matthias Kettl, Nian-Ze Lee, Thomas Lemberger, Marian Lingsch-Rosenfeld, Martin Spiessl, Henrik Wachowitz, and Philipp Wendler. Cpachecker 2.3 with strategy selection: (competition contribution). In *Tools and Algorithms for the Construction and Analysis of Systems: 30th International Conference, TACAS 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6–11, 2024, Proceedings, Part III*, page 359–364, Berlin, Heidelberg, 2024. Springer-Verlag. doi:10.1007/978-3-031-57256-2_21.
3. Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength smt solver. In *Tools and Algorithms for the Construction and Analysis of Systems: 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2–7, 2022, Proceedings, Part I*, page 415–442, Berlin, Heidelberg, 2022. Springer-Verlag. doi:10.1007/978-3-030-99524-9_24.
4. Clark Barrett, Pascal Fontaine, and Cesare Tinelli. The SMT-LIB Standard: Version 2.7. Technical report, Department of Computer Science, The University of Iowa, 2025. Available at www.SMT-LIB.org.
5. Amir M. Ben-Amram and Samir Genaim. On the linear ranking problem for integer linear-constraint loops. *CoRR*, abs/1208.4041, 2012. URL: <http://arxiv.org/abs/1208.4041>, arXiv:1208.4041.
6. Cristina Borralleras, Marc Brockschmidt, Daniel Larraz, Albert Oliveras, Enric Rodríguez-Carbonell, and Albert Rubio. Proving termination through conditional termination. In Axel Legay and Tiziana Margaria, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 99–117, Berlin, Heidelberg, 2017. Springer Berlin Heidelberg.
7. Aaron R. Bradley, Zohar Manna, and Henny B. Sipma. Linear ranking with reachability. In Kousha Etessami and Sriram K. Rajamani, editors, *Computer Aided Verification*, pages 491–504, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
8. Aaron R. Bradley, Zohar Manna, and Henny B. Sipma. *Termination analysis of integer linear loops*, page 488–502. Springer-Verlag, Berlin, Heidelberg, 2005. URL: https://doi.org/10.1007/11539452_37.
9. Mark Braverman. Termination of integer linear programs. In Thomas Ball and Robert B. Jones, editors, *Computer Aided Verification*, pages 372–385, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
10. Marc Brockschmidt, Byron Cook, Samin Ishtiaq, Heidy Khlaaf, and Nir Piterman. T2: Temporal property verification. In Marsha Chechik and Jean-François Raskin, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 387–393, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.
11. Michael Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *International Conference on Computer Aided Verification*, 2003. URL: <https://api.semanticscholar.org/CorpusID:36525>.

12. Michael Colón and Henny Sipma. Synthesis of linear ranking functions. In *Proceedings of the 7th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS 2001, page 67–81, Berlin, Heidelberg, 2001. Springer-Verlag.
13. Byron Cook, Heidy Khlaaf, and Nir Piterman. On automation of ctl* verification for infinite-state systems. In Daniel Kroening and Corina S. Păsăreanu, editors, *Computer Aided Verification*, pages 13–29, Cham, 2015. Springer International Publishing.
14. Byron Cook, Eric Koskinen, and Moshe Vardi. Temporal property verification as a program analysis task. In *Proceedings of the 23rd International Conference on Computer Aided Verification*, CAV’11, page 333–348, Berlin, Heidelberg, 2011. Springer-Verlag.
15. Byron Cook, Andreas Podelski, and Andrey Rybalchenko. Abstraction refinement for termination. In Chris Hankin and Igor Siveroni, editors, *Static Analysis*, pages 87–101, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
16. Patrick Cousot and Radhia Cousot. An abstract interpretation framework for termination. *SIGPLAN Not.*, 47(1):245–258, January 2012. doi:10.1145/2103621.2103687.
17. Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. *ACM Transactions on Programming Languages and Systems*, 13(4):451–490, 1991.
18. Leonardo De Moura and Nikolaj Bjørner. Z3: an efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS’08/ETAPS’08, page 337–340, Berlin, Heidelberg, 2008. Springer-Verlag.
19. Daniel Dietsch, Matthias Heizmann, Vincent Langenfeld, and Andreas Podelski. Fairness modulo theory: A new approach to LTL software model checking. In Daniel Kroening and Corina S. Pasareanu, editors, *Computer Aided Verification - 27th International Conference, CAV 2015, San Francisco, CA, USA, July 18-24, 2015, Proceedings, Part I*, volume 9206 of *Lecture Notes in Computer Science*, pages 49–66. Springer, 2015. doi:10.1007/978-3-319-21690-4_4.
20. Julius Farkas. Theorie der einfachen ungleichungen. *Journal für die reine und angewandte Mathematik (Crelles Journal)*, 1902(124):1–27, 1902.
21. Florian Frohn and Jürgen Giesl. Proving non-termination via loop acceleration. *CoRR*, abs/1905.11187, 2019. URL: <http://arxiv.org/abs/1905.11187>, arXiv:1905.11187.
22. Florian Frohn and Jürgen Giesl. Proving non-termination and lower runtime bounds with loat (system description), 2022. URL: <https://arxiv.org/abs/2202.04546>, arXiv:2202.04546.
23. Jürgen Giesl, Cornelius Aschermann, Marc Brockschmidt, Fabian Emmes, Florian Frohn, Carsten Fuhs, Jera Hensel, Carsten Otto, Martin Plücker, Peter Schneider-Kamp, Thomas Ströder, Stephanie Swiderski, and René Thiemann. Analyzing program termination and complexity automatically with aprove. *Journal of Automated Reasoning*, 58(1):3–31, 2017. doi:10.1007/s10817-016-9388-y.
24. Ashutosh Gupta, Thomas A. Henzinger, Rupak Majumdar, Andrey Rybalchenko, and Ru-Gang Xu. Proving non-termination. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’08, page 147–158, New York, NY, USA, 2008. Association for Computing Machinery. doi:10.1145/1328438.1328459.

25. Matthias Heizmann, Daniel Dietsch, Marius Greitschus, Jan Leike, Betim Musa, Claus Schätzle, and Andreas Podelski. Ultimate automizer with two-track proofs. In *Proceedings of the 22nd International Conference on Tools and Algorithms for the Construction and Analysis of Systems - Volume 9636*, page 950–953, Berlin, Heidelberg, 2016. Springer-Verlag. doi:10.1007/978-3-662-49674-9_68.
26. Matthias Heizmann, Jochen Hoenicke, Jan Leike, and Andreas Podelski. Linear ranking for linear lasso programs. *CoRR*, abs/1401.5347, 2014. URL: <http://arxiv.org/abs/1401.5347>, arXiv:1401.5347.
27. Jera Hensel, Constantin Mensendiek, and Jürgen Giesl. Aprove: Non-termination witnesses for c programs - (competition contribution). In *International Conference on Tools and Algorithms for Construction and Analysis of Systems*, 2022. URL: <https://api.semanticscholar.org/CorpusID:247858517>.
28. Daniel Larraz, Kaustubh Nimkar, Albert Oliveras, Enric Rodríguez-Carbonell, and Albert Rubio. Proving non-termination using max-smt. In Armin Biere and Roderick Bloem, editors, *Computer Aided Verification*, pages 779–796, Cham, 2014. Springer International Publishing.
29. Jan Leike. Ranking function synthesis for linear lasso programs. *ArXiv*, abs/1401.5351, 2014. URL: <https://api.semanticscholar.org/CorpusID:8593497>.
30. Jan Leike and Matthias Heizmann. Ranking templates for linear loops. *Log. Methods Comput. Sci.*, 11(1), 2015. doi:10.2168/LMCS-11(1:16)2015.
31. Jan Leike and Matthias Heizmann. Geometric nontermination arguments. *CoRR*, abs/1609.05207, 2016. URL: <http://arxiv.org/abs/1609.05207>, arXiv:1609.05207.
32. Ravindra Metta, Hrushikesh Karmarkar, Kumar Madhukar, R. Venkatesh, and Supratik Chakraborty. PROTON: PRObes for termination or not (competition contribution). In *TACAS 2024*, volume 14572 of *LNCS*, pages 393–398. Springer, 2024. doi:10.1007/978-3-031-57256-2_27.
33. Ravindra Metta, Umang Mathur, and Saurabh Joshi. PROTON 2.1: Synthesizing ranking functions via fine-tuned locally hosted LLM (competition contribution). In *TACAS 2025*, volume 14682 of *LNCS*, pages 407–411. Springer, 2025.
34. Arkadi Nemirovski and Kees Roos. *Motzkin transposition theorem*. *Motzkin Transposition Theorem*, pages 2345–2348. Springer US, Boston, MA, 2009. doi:10.1007/978-0-387-74759-0_405.
35. Andreas Podelski and Andrey Rybalchenko. A complete method for the synthesis of linear ranking functions. In Bernhard Steffen and Giorgio Levi, editors, *Verification, Model Checking, and Abstract Interpretation*, pages 239–251, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.
36. Caterina Urban. The Abstract Domain of Segmented Ranking Functions. In Logozzo, Francesco, Fähndrich, and Manuel, editors, *Lecture Notes in Computer Science*, volume 7935 of *Lecture Notes in Computer Science*, pages 43–62, Seattle, United States, June 2013. Springer. URL: <https://inria.hal.science/hal-00925670>, doi:10.1007/978-3-642-38856-9_5.
37. Shaowei Zhu and Zachary Kincaid. Reflections on termination of linear loops. In Alexandra Silva and K. Rustan M. Leino, editors, *Computer Aided Verification*, pages 51–74, Cham, 2021. Springer International Publishing.
38. Shaowei Zhu and Zachary Kincaid. Termination analysis without the tears. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2021, page 1296–1311, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3453483.3454110.